

Chapter 2. Kubernetes in a Nutshell

It's helpful to get a quick rundown of what Kubernetes is and how it works if you are new to the space. Many tutorials and 101 courses are available on the web, but I would like to summarize the most important background information and concepts in this chapter. Over the course of this book, I'll reference cluster node components, so feel free to come back to this information at any time.

What Is Kubernetes?

To understand what Kubernetes is, let's first define *microservices* and *containers*.

Microservice architectures call for developing and executing pieces of the application stack as individual services, and those services have to communicate with one another. If you decide to operate those services in containers, you will need to manage a lot of them while at the same time thinking about cross-cutting concerns like scalability, security, persistence, and load balancing.

Tools like [BuildKit](#) and [Podman](#) package software artifacts into a container image. Container runtime engines like [Docker Engine](#) and [containerd](#) use the image to run a container. This works great on developer machines for testing purposes or for ad hoc executions, e.g., as part of a continuous integration (CI) pipeline.

Kubernetes is a container orchestration tool that helps with operating hundreds or even thousands of containers on physical machines, virtual machines, or in the cloud. Kubernetes can also fulfill those cross-cutting concerns mentioned earlier. The container runtime engine integrates with Kubernetes. Whenever a container creation is triggered, Kubernetes will delegate lifecycle aspects to the container runtime engine.

Features

The previous section touched on some features provided by Kubernetes. Here, I am going to dive a little deeper by explaining those features in more detail:

Declarative model

You do not have to write imperative code using a programming language to tell Kubernetes how to operate an application. All you need to do as an end user is to declare a desired state. The desired state can be defined using a YAML or JSON manifest that conforms to an API schema. Kubernetes then maintains the state and recovers it in case of a failure.

Autoscaling

You will want to scale up resources when your application load increases, and scale down when traffic to your application decreases. This can be achieved in Kubernetes by manual or automated scaling. The most practical, optimized option is to let Kubernetes automatically scale resources needed by a containerized application.

Application management

Changes to applications, e.g., new features and bug fixes, are usually baked into a container image with a new tag (likely a version number). You can easily roll out those changes across all containers running them using Kubernetes' convenient replication feature. Kubernetes also allows for rolling back to a previous application version in case of a blocking bug or if a security vulnerability is detected.

Persistent storage

Containers offer only an ephemeral filesystem. Upon restart of the container, all data written to the filesystem is lost. Depending on the nature of your application, you may need to persist data for longer, for example, if your application interacts with a database. Kubernetes offers the ability to mount storage required by application workloads.

To support a microservices architecture, the container orchestrator needs to allow for communication between containers, and from end users to containers from outside of the cluster. Kubernetes employs internal and external load balancing for routing network traffic.

High-Level Architecture

Architecturally, a Kubernetes cluster consists of control plane nodes and worker nodes, as shown in [Figure 2-1](#). Each node runs on infrastructure provisioned on a physical or virtual machine, or in the cloud. The number of nodes you want to add to the cluster and their topology depend on the application's resource needs.

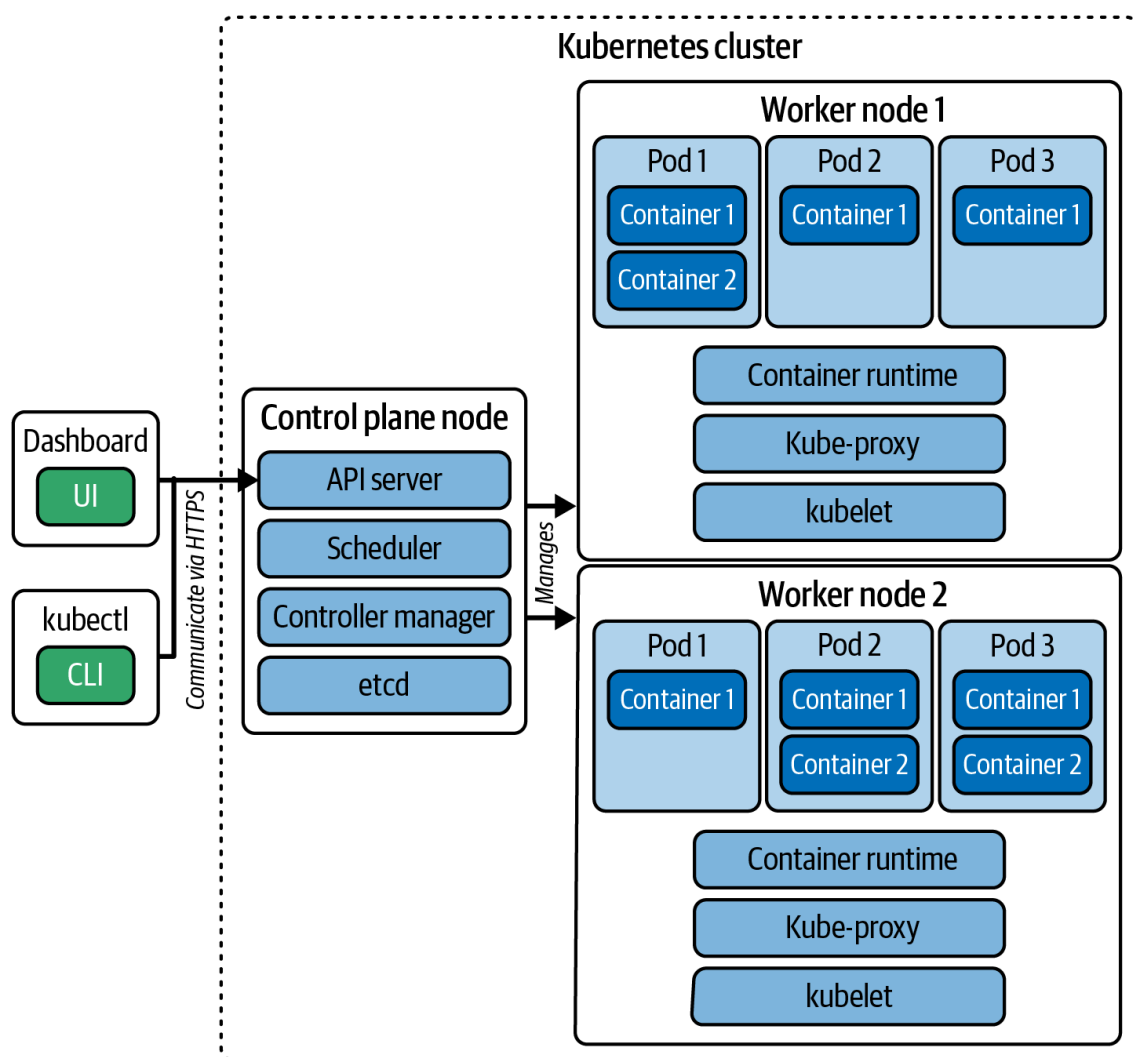


Figure 2-1. Kubernetes cluster nodes and components

Control plane nodes and worker nodes have specific responsibilities:

Control plane node

This node exposes the Kubernetes API through the API server and manages the nodes that make up the cluster. It also responds to cluster events, for example, when the end user requests to scale up the number of Pods to distribute the load for an application.

Production clusters employ a [highly available \(HA\) architecture](#) that usually involves three or more control plane nodes.

Worker node

The worker node executes workload in containers managed by Pods. Every worker node needs a container runtime engine installed on the host machine to be able to manage containers.

In the next two sections, we'll look at the essential components embedded in those nodes to fulfill their tasks. Add-ons like cluster DNS are not discussed explicitly here. See the [Kubernetes documentation](#) for more details.

Control Plane Node Components

The control plane node requires a specific set of components to perform its job. The following list of components will give you an overview:

API server

The API server exposes the API endpoints clients use to communicate with the Kubernetes cluster. For example, if you execute the tool `kubectl`, a command-line based Kubernetes client, you will make a RESTful API call to an endpoint exposed by the API server as part of its implementation. The API processing procedure inside of the API server will ensure aspects like authentication, authorization, and admission control. For more information on that topic, see [Chapter 6](#).

Scheduler

The scheduler is a background process that watches for new Kubernetes Pods with no assigned nodes and assigns them to a worker node for execution.

Controller manager

The controller manager watches the state of your cluster and implements changes where needed. For example, if you make a configuration change to an existing object, the controller manager will try to bring the object into the desired state.

etcd

Cluster state data needs to be persisted over time so it can be reconstructed upon a node restart or even a full cluster restart. That's the responsibility of [etcd](#), an open source software Kubernetes integrates with. At its core, etcd is a key-value store used to persist all data related to the Kubernetes cluster.

Common Node Components

Kubernetes employs components that are leveraged by all nodes independent of their specialized responsibility:

Kubelet

The kubelet runs on every node in the cluster; however, it makes the most sense on a worker node. This is because the control plane node usually doesn't execute workload, and the worker node's primary responsibility is to run workload. The kubelet is an agent that makes sure that the necessary containers are running in a Pod. You could say that the kubelet is the glue between Kubernetes and the container runtime engine and ensures that containers are running and healthy.

Kube-proxy

The kube-proxy is a network proxy that runs on each node in a cluster to maintain network rules and enable network communication. In part, this component is responsible for implementing the Service concept covered in [Chapter 17](#).

Container runtime

As mentioned earlier, the container runtime is the software responsible for managing containers. The kubelet can be configured to choose from a range of different container runtime engines. While you can install a container runtime engine on a control plane, it's

not necessary, as the control plane node usually doesn't handle workload.

Advantages

This section points out some of the most important advantages of Kubernetes, which are summarized here:

Portability

A container runtime engine can manage a container independent of its runtime environment. The container image bundles everything it needs to work, including the application's binary or code, its dependencies, and its configuration. Kubernetes can run applications in a container in on-premise and cloud environments. As an administrator, you can choose the platform you think is most suitable to your needs without having to rewrite the application. Many cloud offerings provide product-specific, opt-in features. While using product-specific features helps with operational aspects, be aware that they will diminish your ability to switch easily between platforms.

Resilience

Kubernetes is designed as a declarative state machine. Controllers are reconciliation loops that watch the state of your cluster, then make or request changes where needed. The goal is to move the current cluster state closer to the desired state.

Scalability

Enterprises run applications at scale. Just imagine how many software components retailers like Amazon, Walmart, or Target need to operate to run their businesses. Kubernetes can scale the number of Pods based on demand or automatically according to resource consumption or historical trends.

API-based

Kubernetes exposes its functionality through APIs. We learned that every client needs to interact with the API server to manage ob-

jects. It is easy to implement a new client that can make RESTful API calls to exposed endpoints.

Extensibility

The API aspect stretches even further. Sometimes, the core functionality of Kubernetes doesn't fulfill your custom needs, but you can implement your own extensions to Kubernetes. With the help of specific extension points, the Kubernetes community can build custom functionality according to their requirements, e.g., monitoring or logging solutions.

Summary

Kubernetes is software for managing containerized applications at scale. Every Kubernetes cluster consists of at least a single control plane node and a worker node. The control plane node is responsible for scheduling the workload and acts as the single entry point to manage its functionality. Worker nodes handle the workload assigned to them by the control plane node.

Kubernetes is a production-ready runtime environment for companies wanting to operate microservice architectures while also supporting non-functional requirements like scalability, security, load balancing, and extensibility.

The next chapter will explain how to interact with a Kubernetes cluster using the command-line tool `kubectl`. You will learn how to run it to manage objects, an essential skill for acing the exam.